MySQL Customer, Product & Sales Analytics Project

15 Major Questions with Hints

Tables: customers (100 rows), products (50 rows), orders (1,000 rows)

1. Database Exploration and Basic Analysis

Using the three tables:

a. Display the structure of each table.
b. Display the number of records in each table.
c. Find the total number of customers, products and orders.
d. Find the average, minimum and maximum product price.
e. Find the average, minimum and maximum order value.

Hint: Use DESCRIBE, COUNT(), AVG(), MIN(), MAX().

2. Filtering and Sorting

Write queries to:

a. Find customers from a particular state.
b. Find products above a specified price.
c. Find products with low stock.
d. Find completed and cancelled orders.
e. Display the 10 largest orders by value.

Hint: Use WHERE, AND, OR, IN, BETWEEN, ORDER BY, and LIMIT.

3. GROUP BY and Aggregate Analysis

Calculate:

a. Number of customers by state.
b. Number of products by category.
c. Number of orders by payment method.
d. Total sales by payment method.

e. Total sales by order status.
f. Average product price by category.

Hint: Combine aggregate functions with GROUP BY.

4. INNER JOIN — Customer Orders

Using INNER JOIN, display:

- **Customer name**
- **Order ID**
- **Order date**
- **Product ID**
- **Quantity**
- **Total amount**

Then calculate the total amount spent by each customer.

Hint: Join customers to orders using customer_id.

5. INNER JOIN — Three-Table Analysis

Join all three tables and produce a detailed sales result containing:

- **Customer name**
- **City**
- **State**
- **Product name**
- **Category**
- **Quantity**
- **Unit price**
- **Total amount**
- **Order date**

- **Payment method**

Then calculate total revenue by product and category.

Hint: You need two JOIN conditions.

6. LEFT JOIN — Customers and Products

Using LEFT JOIN:

a. Display every customer and their orders.
b. Find customers who have never placed an order.
c. Display every product and its orders.
d. Find products that have never been ordered.
e. Calculate total spending for every customer, including customers with zero spending.

Hint: Look for NULL values after the join.

7. RIGHT JOIN, UNION and CROSS JOIN

a. Use RIGHT JOIN to display all products and their orders.
b. Rewrite the same query using LEFT JOIN.
c. Explain the difference between the two approaches.
d. Create a full-outer-join-like result using LEFT JOIN, RIGHT JOIN and UNION.
e. Use CROSS JOIN to generate every possible customer-product combination.

Hint: MySQL does not directly support FULL OUTER JOIN.

8. Customer and Product Performance Analysis

Determine:

a. The top 10 customers by total spending.
b. Customers with more than 10 orders.
c. The top 10 products by revenue.
d. The top 10 products by quantity sold.
e. Products with low stock but high sales.

Hint: Use GROUP BY, ORDER BY, HAVING and aggregate functions.

9. CASE — Customer, Product and Order Classification

Create classifications using CASE.

a. Classify customers as:

Low Spender

Medium Spender

High Spender

b. Classify products according to price.

c. Classify products according to stock level.

d. Classify orders according to order value.

Hint: CASE WHEN ... THEN ... ELSE ... END.

10. Subqueries and CTEs

Use subqueries and/or CTEs to:

a. Find products whose price is above the average product price.
b. Find orders above the average order value.
c. Find customers whose spending is above average.
d. Find the highest-revenue products.
e. Calculate total spending by customer using a CTE.

Hint: First calculate the required average/total in a subquery or CTE, then use the result in the main query.

11. Date and Sales Trend Analysis

Using order_date:

a. Calculate daily sales.
b. Calculate monthly sales.
c. Calculate yearly sales.
d. Find the month with the highest sales.
e. Find the month with the lowest sales.
f. Compare each month's sales with the previous month.

Hint: Use MySQL date functions and LAG()

12. Window Functions and Ranking

Use window functions to:

a. Rank customers by total spending.
b. Rank products by revenue.
c. Number each customer's orders using ROW_NUMBER().
d. Rank products by quantity sold using DENSE_RANK().
e. Calculate each customer's percentage contribution to total sales.

Hint: Use RANK(), DENSE_RANK(), ROW_NUMBER(), LAG() and window SUM().

13. Create a Customer Sales Report Using a Stored Procedure

Create a stored procedure called:

generate_customer_sales_report()

The procedure should create/update a permanent table called:

customer_sales_report

The report should contain:

- **Customer ID**
- **Customer name**
- **City**
- **State**
- **Number of orders**
- **Total quantity purchased**
- **Total amount spent**
- **Average order value**
- **Customer segment**

Hint: Use JOIN, GROUP BY, aggregate functions and CASE.

Important: Use a permanent report table, not a MySQL TEMPORARY TABLE, because the report needs to be accessible through a separate Excel connection.

14. Create Product and Monthly Sales Reports Using Stored Procedures

Create two stored procedures.

Procedure 1

generate_product_sales_report()

Create/update:

product_sales_report

Include:

- **Product ID**
- **Product name**
- **Category**
- **Price**
- **Stock quantity**
- **Units sold**
- **Number of orders**
- **Total revenue**

Procedure 2

generate_monthly_sales_report()

Create/update:

monthly_sales_report

Include:

- **Year**
- **Month**
- **Number of orders**

- **Units sold**
- **Total revenue**
- **Average order value**

Hint: Use INSERT INTO ... SELECT inside the procedures.

15. MySQL → Excel Reporting Project

Connect Excel to the MySQL database and import:

customer_sales_report

product_sales_report

monthly_sales_report

Then answer the following using Excel:

a. Who are the highest-spending customers?
b. Which products generate the highest revenue?
c. Which categories generate the highest sales?
d. Which month generated the highest revenue?
e. Which products have low stock but high sales?
f. Which payment methods generate the most revenue?

Finally:

1. **Add a new order to MySQL.**
2. **Run the appropriate stored procedure.**
3. **Refresh the Excel connection.**
4. **Confirm that the new information appears in Excel.**

The important workflow is:

MySQL Tables

↓

SQL Queries

↓

Stored Procedures

↓

Permanent Report Tables

↓

Excel Connection

↓

Refresh